

Deadline:

- **Deadlines: 11:59 PM Chicago time, on 1st October, 8th October, and 15th October, 2026 (Chicago Clock).**
- The submission link will be closed automatically after the deadline. **No submissions will be accepted after the deadline (including via email).** Please have sufficient time before the deadline to scan your work and upload it to Canvas.

Submission Instructions:

- Total Points: 200
 - Verilog implementation and testbenches (Problems 1–3): 150 points
 - Progress Report 1: 10 points
 - Progress Report 2: 10 points
 - Final Report: 30 points
- Optional Bonus Points: 100

Simulation Environment: For this project, students should use [EDA Playground](#) to write, compile, and simulate their Verilog designs. Waveform screenshots generated through EDA Playground should be included in the final report where required.

Resource for Verilog Hardware Description Language: ASIC World is a concise resource for Verilog description language. You can find it [here](#). In addition to the Verilog tutorial that Suresh provided and the Verilog primer that was discussed in the class, please look at the following modules before you embark into the project.

1. **Verilog in One Day:** [Click Here](#).
2. **Verilog Operators:** [Click Here](#).
3. **Behavioral Modeling:** [Click Here](#).
4. **Structural and Gate-level Modeling:** [Click Here](#).
5. **Testbench Writing:** [Click Here](#).

1 Problem Sets

Problem 1 (Designing a 1-bit/4-bit Full Adder/Subtractor). Refer to Lecture Slide 2 for the following problems.

- (a) Design a 1-bit full-adder in Verilog using *behavioral modeling*. Assume that the inputs are **A**, **B**, and **Cin**, and outputs are **S**, **Cout**. Use the module template of Figure 1(a). [Points : 5]

```

1 module one_bit_full_adder(A, B, Cin, S,
  Cout);
2
3 input A, B, Cin;
4 output S, Cout;
5
6 // Your code
7
8 endmodule

```

(a) Module definition of 1-bit full adder.

```

1 module four_bit_RCA_RCS(A, B, Cin, S,
  Cout);
2
3 input [3:0] A, B;
4 input Cin;
5 output [3:0] S;
6 output Cout;
7
8 // Your code
9
10 endmodule

```

(b) Module definition of 4-bit RCA/RCS.

Figure 1: Module definitions for Problem 1.

- (b) Design a 1-bit full-adder in Verilog using *structural modeling*. Assume that the inputs are **A**, **B**, and **Cin**, and outputs are **S**, **Cout**. Use the module template of Figure 1(a). [Points : 5]
- (c) Using the 1-bit full adder from **either** Part (a) or Part (b), design a 4-bit Ripple-Carry Adder (RCA). Use the module template of 1(b). [Points : 10]
- (d) Enhance the 4-bit RCA of Part (c) to construct 4-bit Ripple-Carry Subtractor (RCS). Use the module template of 1(b). [Hint: Look at Lecture Slide 2, Slide 18.] [Points : 10]
- (e) Design a testbench to test the 4-bit RCA/RCS. The testbench must include at least the following cases:
- one unsigned addition,
 - one unsigned subtraction,
 - one signed addition involving a negative operand,
 - one signed subtraction involving a negative operand, and
 - one case that produces a carry-out.

For signed operations, interpret the operands using two's-complement representation. Additional test cases may be included if desired. Clearly show the input values and corresponding output waveforms. [Points : 20]

Problem 2 (Designing a 32-bit CLA Adder/Subtractor). Refer to Lecture Slide 2 for the following problems.

- (a) Design a 32-bit Carry Lookahead Adder (CLA) using a *block size of four* (4). Assume that the inputs are **A[31:0]**, **B[31:0]**, and **Cin**, and outputs are **S[31:0]**, **Cout**. Use the 4-bit full adder (RCA) that you designed in Problem 1 as the component. You can only use **2-input AND** and **OR** gates to construct the carry propagation and carry generate logic. **No need to include subtraction operation for the CLA.** Use the CLA module template of Figure 2(a). [Points : 30]

```

1 module CLA(A, B, Cin, S, Cout);
2
3 input [31:0] A, B;
4 input Cin;
5 output [31:0] S;
6 output Cout;
7
8 // Your code
9
10 endmodule

```

(a) Module definition of 32-bit CLA.

```

1 module PPA(A, B, Cin, S, Cout);
2
3 input [15:0] A, B;
4 input Cin;
5 output [15:0] S;
6 output Cout;
7
8 // Your code
9
10 endmodule

```

(b) Module definition of 16-bit PPA.

Figure 2: Module definitions for Problem 2 and Problem 3.

- (b) Enhance the 4-bit RCA/RCS testbench of Problem 1(e) to test the 32-bit CLA. In addition to the previous types of test cases, include at least one test case that requires carry propagation across multiple 4-bit blocks. [Points : 20]

Problem 3 (Designing a 16-bit Prefix Adder (PPA)). Refer to the textbook "Digital Design and Computer Architecture, By Harris and Harris, 2nd edition", Section 5.2.1 "Addition", Subsection "Prefix Adders." The textbook is available online and also for free at [Digital Design and Computer Architecture](#). An example 16-bit Parallel Prefix Adder is given in Figure 3. Here $P_{-1} = 0$ and $G_{-1} = C_{in}$. You can extend this to make a 32-bit PPA.

- (a) Design a 16-bit PPA. Assume that the inputs are **A[15:0]**, **B[15:0]**, and **Cin**, and outputs are **S[15:0]**, **Cout**. You can only use **2-input AND** and **OR** gate to construct the carry propagation and carry generate logic. Use the PPA module template of Figure 2(b) [Points : 40]
- (b) Enhance the CLA testbench of Problem 2(b) to test the 16-bit PPA. [Points : 10]

Hints :

- Figure 3 mentions the circuit for each block.
- The solid white square blocks on the top of the diagram is often known as the "pre-computation block". They calculate the "bit-level" generate and propagate signals.

$$P_{i:i} = A_i \mid B_i$$

$$G_{i:i} = A_i \& B_i$$

- The curved-corner square blocks in the bottom of the diagram is often known as the "post-computation block". They calculate the final sum bit.

$$S_i = G_{(i-1):-1} \oplus (A_i \oplus B_i)$$

- The solid black square blocks in between the pre-computation and post-computation blocks calculates the "block-level" generate and propagate signals.

$$P_{i:j} = P_{i:k} \& P_{(k-1):j}$$

$$G_{i:j} = G_{i:k} \mid (P_{i:k} \& G_{(k-1):j})$$

Rev: 09/16/2026

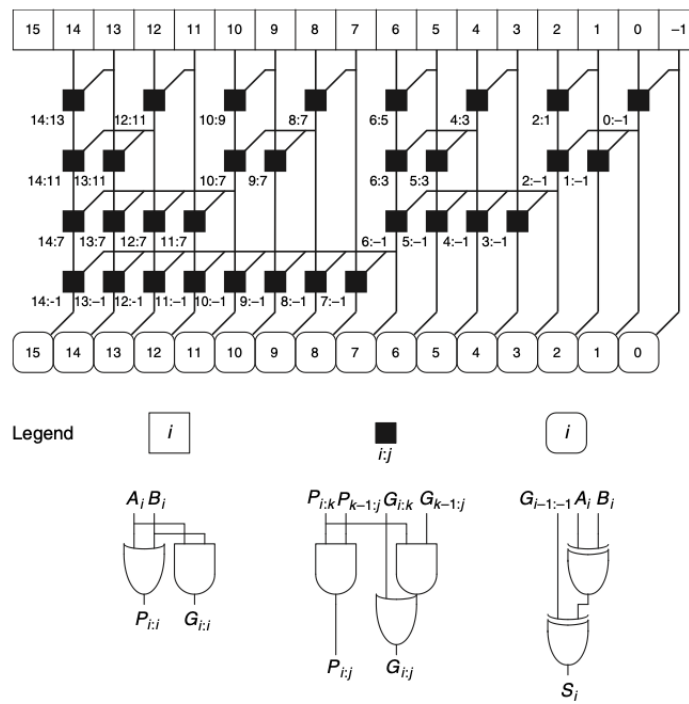


Figure 3: A 16-bit Parallel Prefix Adder.

- The "-1" input refer's to the "cin". Here,

$$P_{ii} = 0$$

$$G_{ii} = cin$$

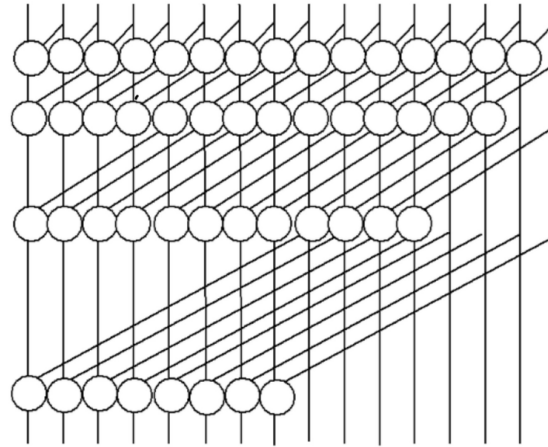


Figure 4: The computation tree structure of a 16-bit Kogge-Stone (KS) Adder.

The following problem is a bonus problem and optional. If you solve this, you will earn additional points.

Problem 4 (Designing a 16-bit Kogge-Stone (KS) Adder). Please refer to the research article titled "A Taxonomy of Parallel Prefix Networks" by David Harris. You can find it at [Kogge-Stone \(KS\)](#). You can also refer to Figure 4. This is similar to the PPA in Figure 3. You might have to change $P_{i:i}$ as $P_{i:i} = A_i \oplus B_i$.

- Design a 16-bit KS Adder. Assume that the inputs are **A[15:0]**, **B[15:0]**, and **Cin**, and outputs are **S[15:0]**, **Cout**. You can only use **2-input AND** and **OR** gate to construct the carry propagation and carry generate logic. [Points : 75]
- Enhance the CLA testbench of Problem 2(b) to test the 16-bit KS Adder. [Points : 25]

Hints :

- In the linked paper, Figure-2(c) shows the block diagram of a 16-bit kogge-stone adder. The 8-sided polygon on the top of the circuit is the pre-computation block, which generates the bit-level propagate and generate signals. Although the block is drawn as single long box, each bit needs an individual bit-level generate and propagate signal computation.

$$P_{i:i} = A_i \oplus B_i$$

$$G_{i:i} = A_i \& B_i$$

- The rectangle in the bottom of the diagram is the post-computation block. It calculates the sum, bit by bit. Therefore each bit of the sum needs an individual computation circuit, even though the rectangle is drawn across all bits.

$$S_i = G_{(i-1):-1} \oplus (A_i \oplus B_i)$$

$$S_i = G_{(i-1):-1} \oplus P_{i:i}$$

- The black boxes in the diagram calculates the block-level generate and propagate signals. The following equation is used.

$$P_{i:j} = P_{i:k} \& P_{(k-1):j}$$

$$G_{i:j} = G_{i:k} \mid (P_{i:k} \& G_{(k-1):j})$$

- The gray boxes calculate only the block-level generate signals. As some calculations do not need both P_x and G_x , we can minimize the circuit by only calculating G_x .

$$G_{i:j} = G_{i:k} \mid (P_{i:k} \& G_{(k-1):j})$$

- The white triangle are buffer blocks. They simply pass the input to the output.

$$Output = input$$

2 Submission Deadlines, Expectations, and Artifacts

Create a GitHub repository for your group. The GitHub repo will be used for collaborative coding by your team members. Commit and push your codes as frequently as possible. The commit history may be used as an indication of the contribution of each of the team members. A nice GitHub cheat sheet is available [here](#).

Grading of Code and Reports: The point values listed in Problems 1–3 are awarded based on the correctness and required implementation of the submitted Verilog modules and testbenches. The progress reports and final report are graded separately. The final report should document the group's work, briefly explain the implemented designs, identify team member responsibilities, and provide simulation waveform screenshots demonstrating the functionality of the designs.

- (A) **Progress Report 1 [10 Points]:** Complete the progress report with the following details – (i) Summary of Project Status and (ii) Team Member Responsibilities. Submit a PDF file and include the GitHub link in the PDF file.

- **Deadline:** 10/01/2026 at 11:59 pm Chicago Time.
- **Expectation:** Completion of Problem 1 and initiation of Problem 2.

- (B) **Progress Report 2 [10 Points]:** Complete the progress report with the following details – (i) Summary of Project Status and (ii) Team Member Responsibilities. Submit a PDF file and include the GitHub link in the PDF file.

- **Deadline:** 10/08/2026 at 11:59 pm Chicago Time.

- **Expectation:** Completion of Problem 2 and initiation of Problem 3.

(C) **Final Report [30 Points]:** Submit a final report containing:

- (i) a brief explanation of the design and implementation of each problem,
- (ii) team member responsibilities,
- (iii) screenshots of simulation waveforms for each required design, and
- (iv) the GitHub repository link.

Submit all Verilog source files and testbenches as a separate tarball ([How to tar?](#)). We may pull code from your GitHub; ensure that the submitted code and GitHub code match and are the latest versions.

- **Deadline:** 10/15/2026 at 11:59 pm Chicago Time.
- **Expectation:** Completion of all mandatory problems and (optional) bonus problem.

Rev: 09/16/2026

**NO EXTENSION OF ANY DEADLINE WILL BE GRANTED
UNDER ANY CIRCUMSTANCES.**